UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2025

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Examen U.2
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	1ro Software B
Alumnos participantes:	Andrade Sánchez Hugo Alexander Pomaquero Chango Katherine Soledad Bedoya Mazo Juan Gabriel
Asignatura:	Algoritmos y Lógica de programación
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE EVALUACION

2.1 Objetivos

General:
Desarrollar un sistema en C++ que gestione vehículos y viajes mediante clases, constructores y validaciones básicas, integrando un menú interactivo y el módulo obligatorio del juego del ahorcado, con evidencia del proceso metodológico y versionamiento en GitHub Desktop y GitHub.

Específicos:

- Implementar las clases Vehiculo y Viaje con atributos privados, constructores, métodos de acceso y validaciones simples, permitiendo registrar y mostrar información de manera estructurada**.**
- Diseñar un menú principal que integre las opciones de registro y consulta de vehículos y viajes, junto con el acceso al módulo del juego del ahorcado, validando entradas y asegurando la interacción básica del usuario.
- Gestionar el proyecto en **GitHub Desktop y GitHub**, realizando commits parciales con mensajes significativos, publicando el repositorio con un README que documente el análisis, la metodología seguida y las partes implementadas.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 1
No presenciales: 0

2.4 Instrucciones

Este examen integra los contenidos desarrollados en el parcial y en el proyecto: análisis de problemas, estructuras de control, validación de datos, funciones, arreglos o vectores, recursividad, programación orientada a objetos, encapsulamiento, constructores, depuración y defensa técnica.

Cada grupo mantiene su caso principal asignado, pero ahora todos deben resolver además un módulo obligatorio de Ahorcado, sin recibir el código fuente como solución. El código cargado por el docente se usa únicamente como referencia funcional para construir el enunciado y elevar el nivel de razonamiento del examen.

La intención es que el estudiante piense, diseñe, implemente, pruebe, documente y evidencie su proceso de desarrollo real en C++ desde Windows, utilizando GitHub Desktop y un repositorio publicado en GitHub.

2.5 Listado de equipos, materiales y recursos

Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.
- Papel
- Material de escritorio
-

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): Lenguaje de programación C++,IDE para programar(VS)

2.6 Actividades por desarrollar

-Definir clases Vehiculo y Viaje con atributos, constructores y validaciones básicas.

-Implementar menú principal con opciones de registro, consulta y búsqueda.

--Integrar el módulo obligatorio del juego del ahorcado en el mismo programa.

-Realizar pruebas de escritorio y depuración en Visual Studio Code.

-Gestionar el proyecto en GitHub Desktop con mínimo 4 commits significativos.

-Publicar el repositorio en GitHub con README documentando metodología y partes implementadas.

-Presentar evidencias de commits, ejecución funcional y defensa técnica.

2.7 Resultados obtenidos

Se implementó correctamente el sistema en C++ con las clases Vehiculo y Viaje, incluyendo constructores, validaciones y métodos de consulta.

El menú principal permite registrar y mostrar vehículos y viajes, además de calcular el rendimiento de cada viaje.

Se integró el módulo obligatorio del juego del ahorcado, funcionando con validación de entradas y vocabulario del grupo.

Se realizaron pruebas de escritorio y ejecución en Visual Studio Code, verificando el correcto registro y visualización de datos.

El proyecto fue gestionado en GitHub Desktop, con al menos 4 commits significativos que evidencian el proceso de desarrollo.

El repositorio fue publicado en GitHub con un README que documenta metodología, caso asignado y partes implementadas.

Se cuenta con evidencias de commits, ejecución funcional y defensa técnica, cumpliendo los criterios de la rúbrica de evaluación

2.8 Habilidades blandas empleadas en la práctica

☒ Liderazgo
☒ Trabajo en equipo
☒ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

-El sistema desarrollado en C++ cumple con la gestión de vehículos y viajes, aplicando clases, constructores y validaciones básicas.

-El menú principal permite registrar y consultar información de manera ordenada, mostrando resultados claros al usuario.

-Se integró el módulo obligatorio del juego del ahorcado, garantizando la funcionalidad solicitada en la rúbrica.

-El uso de GitHub Desktop y GitHub permitió evidenciar el proceso metodológico mediante commits parciales y un repositorio documentado.

2.10 Recomendaciones

-Mantener una estructura modular del código para facilitar futuras ampliaciones y correcciones.

-Realizar pruebas de escritorio antes de cada commit para asegurar la estabilidad del sistema.

-Documentar cada fase del desarrollo en el README para fortalecer la defensa técnica.

-Seguir practicando con GitHub Desktop y GitHub para consolidar hábitos de versionamiento y trabajo colaboratis

2.11 Anexos

Análisis del problema

El sistema debe resolver la gestión de vehículos y viajes de un grupo de transporte. Se requiere:

- Registrar vehículos con datos básicos (placa, tipo, capacidad de tanque, kilometraje, estado).
- Registrar viajes asociados a un vehículo (ruta, distancia, litros abastecidos y consumidos, chofer).
- Mostrar listados de vehículos y viajes.
- Ordenar viajes por rendimiento (km/l).
- Buscar información por placa y chofer (incluyendo búsqueda recursiva).
- Generar reportes de viajes críticos con bajo rendimiento (< 5 km/l).
- Integrar un módulo de juego del Ahorcado, que valide letras y muestre intentos hasta ganar o perder.

El problema combina gestión de datos con interacción de usuario mediante un menú, aplicando programación orientada a objetos, validaciones, ordenamiento, búsqueda y recursividad.

Algoritmo (Entrada, Proceso, Salida)

Entrada:

- Datos de vehículos: placa, tipo, capacidad, kilometraje, estado.
- Datos de viajes: ruta, distancia, litros abastecidos, litros consumidos, chofer, placa del vehículo.
- Letras ingresadas por el usuario en el juego del ahorcado.
- Opciones seleccionadas en el menú.

Proceso:

- Validar datos ingresados (ej. placa no vacía, kilometraje ≥ 0, litros consumidos ≤ abastecidos).
- Almacenar vehículos y viajes en vectores.
- Calcular rendimiento de viajes (distancia / litros consumidos).
- Ordenar viajes por rendimiento con burbuja.
- Buscar vehículos y viajes por placa.
- Buscar viajes por chofer usando recursividad.
- Generar reporte de viajes críticos con rendimiento bajo.
- Ejecutar el juego del ahorcado con intentos limitados y actualización del progreso.

Salida:

- Listado de vehículos registrados.
- Listado de viajes registrados con rendimiento.
- Viajes ordenados por rendimiento.
- Resultados de búsqueda por placa o chofer.
- Reporte de viajes críticos.
- Mensajes de victoria o derrota en el juego del ahorcado

Pseudocodigo

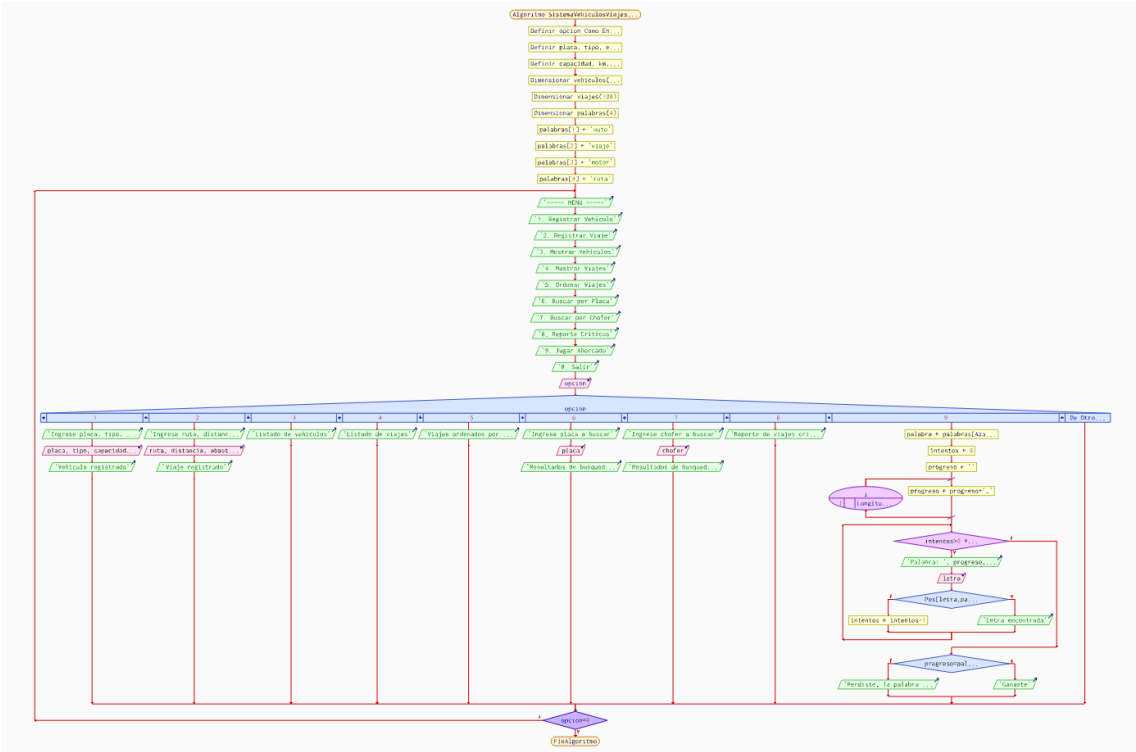
```
Proceso SistemaVehiculosViajesAhorcado
   Definir vehiculos Como Vector
   Definir viajes Como Vector
   Definir palabras Como Vector = ["auto","viaje","motor","ruta"]
   Definir opcion Como Entero

   Repetir
      Mostrar "===== MENU ====="
      Mostrar "1. Registrar Vehiculo"
      Mostrar "2. Registrar Viaje"
      Mostrar "3. Mostrar Vehiculos"
      Mostrar "4. Mostrar Viajes"
      Mostrar "5. Ordenar Viajes"
      Mostrar "6. Buscar por Placa"
      Mostrar "7. Buscar por Chofer"
      Mostrar "8. Reporte Criticos"
      Mostrar "9. Jugar Ahorcado"
      Mostrar "0. Salir"
      Leer opcion

      Segun opcion Hacer
         Caso 1:
            Leer placa, tipo, capacidad, km, estado
            Agregar Vehiculo(placa,tipo,capacidad,km,estado) a vehiculos
         Caso 2:
            Leer ruta, distancia, abastecido, consumido, chofer, placa
            Agregar Viaje(ruta,distancia,abastecido,consumido,chofer,placa) a viajes
```

```
        Caso 3:
          Para cada v en vehiculos Hacer
            Mostrar v
          FinPara
        Caso 4:
          Para cada v en viajes Hacer
            Mostrar v
          FinPara
        Caso 5:
          Ordenar viajes por rendimiento
        Caso 6:
          Leer placa
          Buscar en vehiculos y viajes por placa
        Caso 7:
          Leer chofer
          Buscar recursivamente en viajes por chofer
        Caso 8:
          Mostrar viajes con rendimiento < 5
        Caso 9:
          palabra <- Aleatorio(palabras)
          intentos <- 6
          progreso <- "_"
          Mientras intentos > 0 Y progreso != palabra Hacer
            Mostrar progreso e intentos
            Leer letra
            Si letra está en palabra Entonces
              Actualizar progreso
            SiNo
              intentos <- intentos - 1
            FinSi
          FinMientras
          Si progreso = palabra Entonces
            Mostrar "Ganaste"
          SiNo
            Mostrar "Perdiste, la palabra era: ", palabra
          FinSi
      FinSegun
    Hasta Que opcion = 0
FinProceso
```

DIAGRAMA DE FLUJO



CODIGO EN GITHUB
https://github.com/Kathy2070/ExamenP1_cpp_GrupoB/blob/main/EvaluacionGB

PRUEBA DE ESCRITORIO

```
Palabra: _ _ _ _ _
Intentos: 5
e

Palabra: _ _ _ _ _
Intentos: 4
i

Palabra: _ _ _ _ _
Intentos: 3
o

Palabra: _ o _ o _
Intentos: 3
a

Palabra: _ o _ o _
Intentos: 2
u

Palabra: _ o _ o _
Intentos: 1
t

Palabra: _ o t o _
Intentos: 1
j
Perdiste! Era: motor

===== MENU =====
1. Registrar Vehiculo
2. Registrar Viaje
3. Mostrar Vehiculos
4. Mostrar Viajes
5. Ordenar Viajes
6. Buscar por Placa
7. Buscar por Chofer
8. Reporte Criticos
9. Jugar Ahorcado
0. Salir
```